

Informe de Modernización del Proyecto Crowd-TA-UVE

Estiven Martínez

19 de agosto de 2026

Resumen

El presente informe documenta el proceso completo de modernización del proyecto *“The Crowd Thinks Aloud”* (Crowd-TA-UVE), realizado entre el 18 y 19 de agosto de 2026. El proyecto fue migrado desde Django 3.0.8 y Python 3.8 a Django 5.1 y Python 3.13, eliminando dependencias obsoletas, adaptando la configuración para entornos Windows, creando la base de datos SQLite, y poniendo en funcionamiento el servidor de desarrollo. Se incluyen todas las etapas, desde la actualización del entorno hasta la fusión de ramas en Git y la puesta en marcha del flujo completo del proyecto.

1. Introducción

El proyecto *Crowd-TA-UVE* fue desarrollado originalmente con Django 3.0.8 y Python 3.8, y dependía de varias librerías que hoy en día están obsoletas o son incompatibles con las versiones modernas de Python (3.13). El objetivo de esta modernización fue hacer que el proyecto pudiera ejecutarse en el entorno actual del desarrollador (Windows 11, Python 3.13) sin errores, manteniendo la funcionalidad original y preparando el código para futuras mejoras.

El proceso se dividió en 7 etapas, documentadas a continuación.

2. Etapa 1: Actualización del Entorno y Dependencias

2.1. Creación del Entorno Virtual

Se creó un entorno virtual aislado utilizando Python 3.13.0, asegurando que todas las instalaciones futuras no afecten al sistema global.

```
1 cd C:\Users\ESTIVEN MARTINEZ\Desktop\Practica\crowd-ta-uve\Source_Code
2 python -m venv venv
3 .\venv\Scripts\activate
```

Listing 1: Comandos para crear y activar el entorno virtual

2.2. Actualización de Pip

```
1 python -m pip install --upgrade pip
```

Listing 2: Actualización de pip

2.3. Instalación de Django 5.1

```
1 pip install django==5.1
```

Listing 3: Instalación de Django 5.1

2.4. Instalación de Dependencias Modernas

```
1 pip install django-crispy-forms>=2.1
2 pip install crispy-bootstrap5>=0.7
3 pip install django-cors-headers>=4.3
4 pip install django-extensions>=3.2
5 pip install django-countries>=7.6
6 pip install pillow>=10.0
```

Listing 4: Instalación de dependencias actualizadas

2.5. Reemplazo de django-gsheets por gspread

La librería `django-gsheets` (y su dependencia `gsheets`) estaban abandonadas y no son compatibles con Django 5.1. Se decidió reemplazarlas por las librerías estándar para interactuar con Google Sheets:

```
1 pip install gspread>=6.0
2 pip install google-api-python-client>=2.0
3 pip install google-auth-oauthlib>=1.0
```

Listing 5: Instalación de nuevas librerías para Google Sheets

2.6. Generación de requirements_modern.txt

```
1 pip freeze > requirements_modern.txt
```

Listing 6: Exportación de dependencias

2.7. Gestión de Versiones con Git

```
1 git init
2 git add .
3 git checkout -b ramaEstiven
4 git commit -m "Estado inicial del proyecto antes de la modernización (
    ramaEstiven)"
5 git remote add origin https://github.com/estiradikal/crowd-ta-UVE
6 git push -u origin ramaEstiven
```

Listing 7: Comandos Git ejecutados

3. Etapa 2: Actualización de settings.py

Se modificó el archivo `settings.py` para hacerlo compatible con Django 5.1 y con el entorno Windows.

3.1. Cambios realizados

- `DEBUG = True` (para desarrollo local).
- `ALLOWED_HOSTS = ['localhost', '127.0.0.1']`.
- Base de datos SQLite configurada.
- Rutas de archivos estáticos adaptadas a Windows.
- Se agregó `CorsMiddleware` al principio de `MIDDLEWARE`.
- Se agregó `'django_language_field'` a `INSTALLED_APPS`.
- Se agregó `DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'`.
- Se eliminó la opción obsoleta `USE_L10N`.

4. Etapa 3: Eliminación de Dependencias Obsoletas

4.1. Reemplazo de `django-language-field` por `CharField`

El paquete `django-language-field` no se instalaba correctamente, por lo que se decidió reemplazar el campo `LanguageField` en los modelos por un `CharField` estándar.

- En `models.py`: `mother_tongue = models.CharField(max_length=32, null=False, default="unknown")`
- En `forms.py`: `mother_tongue = forms.CharField(max_length=32, required=True, label="Mother Tongue")`
- Se eliminó `'django_language_field'` de `INSTALLED_APPS`.

4.2. Eliminación de `gsheets.mixins`

Las clases `TaskStatus` y `Approved_Testers` ya no heredan de los mixins de `gsheets`. Se eliminó la dependencia completamente.

- Se comentó `from gsheets import mixins` en `models.py`.
- Se eliminó la herencia de `mixins.SheetPushableMixin` y `mixins.SheetSyncableMixin`.
- Se comentó la URL `path('', include('gsheets.urls'))` en `urls.py`.
- Se eliminó `'gsheets'` de `INSTALLED_APPS`.

5. Etapa 4: Migraciones y Base de Datos

5.1. Creación de la Base de Datos

Se eliminó el archivo `db.sqlite3` (si existía) y se ejecutaron las migraciones para crear las tablas desde cero.

```
1 del db.sqlite3
2 python manage.py makemigrations
3 python manage.py migrate usabilityTest
4 python manage.py migrate
```

Listing 8: Comandos para migrar la base de datos

5.2. Poblado de Datos Iniciales

Se crearon dos tareas en la tabla `TasksDescription` para poder probar el flujo completo del proyecto.

```
1 from usabilityTest.models import TasksDescription
2
3 TasksDescription.objects.create(
4     task_id='1',
5     task_description='Encuentra el curso "Global Warming Science" en la
6     plataforma EDX.',
7     task_valid_question='Cul es el nombre de uno de los instructores?',
8     valid_ans_options=['Prof. John Smith', 'Dr. Jane Doe'],
9     answer='John Smith'
10 )
11
12 TasksDescription.objects.create(
13     task_id='2',
14     task_description='Encuentra cursos avanzados sobre "Climate change".',
15     task_valid_question='Cul es el nombre de uno de los cursos?',
16     valid_ans_options=['Climate Change Science', 'Energy Systems'],
17     answer='Climate Change Science'
```

Listing 9: Código para insertar tareas de prueba

6. Etapa 5: Corrección de Errores en `views.py`

6.1. Descomentado de Variables Globales

Se descomentaron las líneas que definían las variables globales `Total_Tasks_`, `Total_Active`, `Task_1_Answer` y `Task_2_Answer` para corregir el error `NameError` en la vista `payment_id`.

```
1 Total_Tasks_ = TasksDescription.objects.all().count()
2 Total_Active = SubjectProfile.objects.all().count()
3 Task_1_Answer = TasksDescription.objects.get(task_id=1).answer
4 Task_1_Answer = Task_1_Answer.split("|")
5 Task_2_Answer = TasksDescription.objects.get(task_id=2).answer
6 Task_2_Answer = Task_2_Answer.split("|")
```

Listing 10: Variables descomentadas en `views.py`

6.2. Corrección de Importación de redirect

Se agregó la importación de `redirect` en `urls.py` para corregir el error `NameError` en la redirección de la raíz.

```
1 from django.shortcuts import redirect
```

Listing 11: Importación de redirect

7. Etapa 6: Configuración de Crispy Forms con Bootstrap 4

El proyecto original usaba Bootstrap 4, pero se había instalado `crispy-bootstrap5`. Se corrigió instalando `crispy-bootstrap4` y ajustando `settings.py`.

```
1 pip uninstall crispy-bootstrap5 -y
2 pip install crispy-bootstrap4
```

Listing 12: Cambio a Bootstrap 4

```
1 INSTALLED_APPS = [
2     ...
3     'crispy_forms',
4     'crispy_bootstrap4',
5     ...
6 ]
7
8 CRISPY_ALLOWED_TEMPLATE_PACKS = ("bootstrap4",)
9 CRISPY_TEMPLATE_PACK = "bootstrap4"
```

Listing 13: Configuración en settings.py

8. Etapa 7: Gestión de Ramas en Git y Fusión

8.1. Creación de la Rama master

Se creó la rama `master` a partir de la rama remota `origin/master` (que ya existía en GitHub como rama por defecto).

```
1 git fetch origin
2 git checkout -b master origin/master
```

Listing 14: Creación de master local

8.2. Eliminación de la Rama main

Se eliminó la rama `main` local y remota para mantener solo `master` como rama principal.

```
1 git branch -D main
2 git push origin --delete main
```

Listing 15: Eliminación de main

8.3. Fusión de ramaEstiven en master

Se fusionaron todos los cambios modernizados de `ramaEstiven` en `master` usando la bandera `-allow-unrelated-histories` para permitir la fusión de historias diferentes.

```
1 git checkout master
2 git merge ramaEstiven --allow-unrelated-histories
3 git push origin master
```

Listing 16: Fusión de ramaEstiven en master

9. Estado Actual del Proyecto

Tras completar todas las etapas, el proyecto se encuentra en el siguiente estado:

- **Entorno:** Python 3.13 con Django 5.1, entorno virtual `venv`.
- **Dependencias:** Todas las librerías actualizadas y compatibles.
- **Base de datos:** SQLite con migraciones aplicadas y datos iniciales (2 tareas).
- **Código:** Adaptado a Django 5.1, sin dependencias obsoletas (`gsheets`, `django-language-field`).
- **Git:** Rama `master` como rama principal, con todo el código modernizado.
- **Servidor:** Funcionando localmente en `http://127.0.0.1:8000/` con redirección a `/index/?workerId=test&campId=test`.
- **Flujo:** Registro de usuarios, entrenamiento, tareas y pago funcionando correctamente.

10. Próximos Pasos

- Probar el flujo completo en diferentes navegadores.
- Ajustar el campo `Mother Tongue` para que tenga opciones predefinidas (si se desea).
- Implementar la sincronización con Google Sheets usando `gspreadd` (actualmente comentada).
- Configurar variables de entorno para secretos y claves de API.
- Preparar el proyecto para producción (cambiar `DEBUG = False`, configurar servidor WSGI/ASGI).

11. Conclusiones

La modernización del proyecto *Crowd-TA-UVE* se ha completado con éxito. El proyecto ahora es ejecutable en el entorno actual del desarrollador con Django 5.1 y Python 3.13, sin errores de sintaxis o dependencias obsoletas. La creación de una rama `master` estable permite que otros colaboradores puedan clonar y trabajar sobre el código sin problemas. El flujo completo del proyecto (registro, tareas, pago) ha sido probado y funciona correctamente en el entorno de desarrollo local.